

저가 항공사의 프리미엄 서비스 도입 효과 분석

이유정 이현정 김재환 이서연

• INDEX

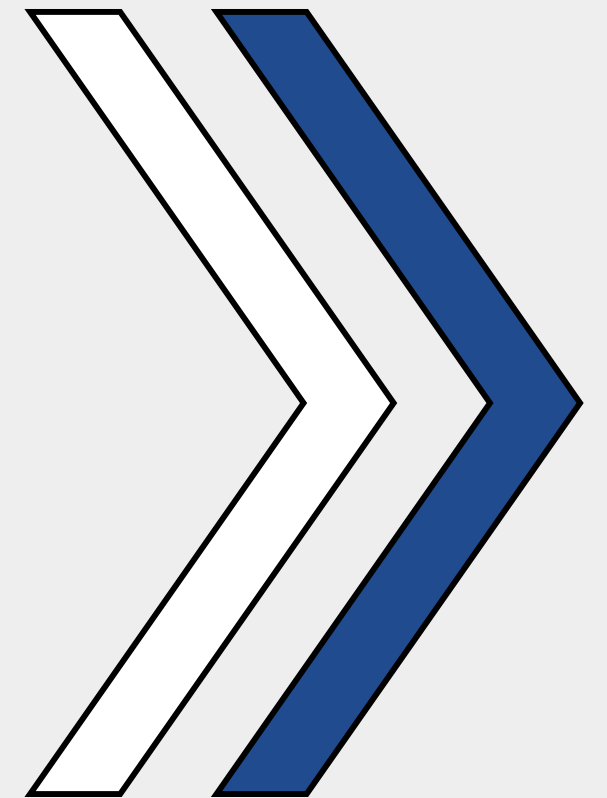
목차

01	문제 정의 및 분석 목표
02	EDA
03	Feature Engineering
04	모델링 및 진단
05	최적화 및 시뮬레이션
06	결과 해석 및 결론



01

문제 정의 및 분석 목표



- 주제 선정 배경
- 분석 목표 및 접근 방법

01

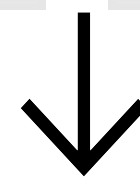
문제 정의

인도 항공업계의 대조적 흐름

- Indigo : 매출 급증, 점유율 확대
- Air India : 민영화 후 투자 확대
- 다수 LCC (Go First 등) : 파산

항공 산업의 구조적 문제

- 치열한 가격 경쟁 심화
- 주요 노선 평균 79달러 (한화 약 10만원) 수준



핵심 문제의식

단순 가격 경쟁만으로는
저가항공사의 지속 가능성 확보 어려움

02

주제 선정 이유

- 단순 가격경쟁 → 지속 불가능
- Indigo 항공: 비즈니스석 도입 후 매출 성장
- LCC도 프리미엄 서비스 도입 시 수익성 개선 가능

03

분석 목표

- 데이터

fsc_df: 비즈니스 + 이코노미
(학습용)

lcc_df: 이코노미 전용
(적용 대상)

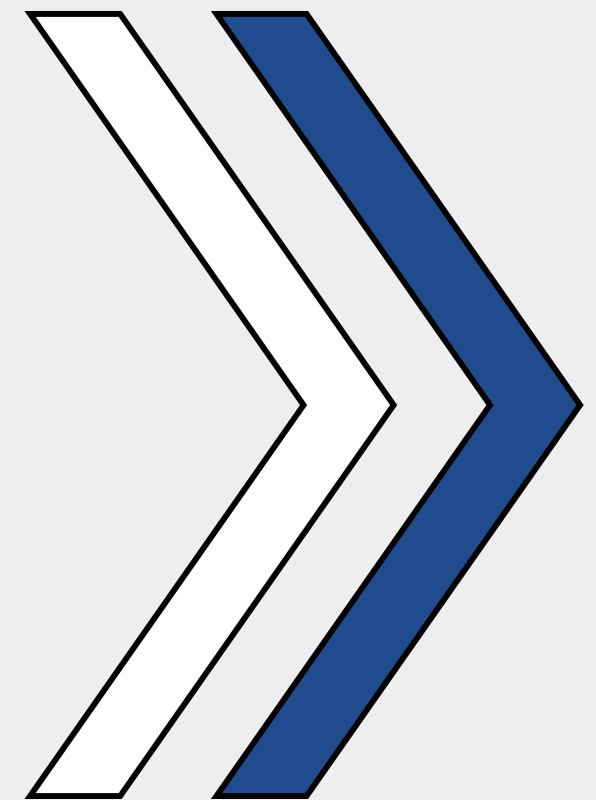
- 접근 방법

fsc_df로 모델 학습 → lcc_df 적용
각 항공편별 최적 비즈니스석 수 도출
예측 수익성 및 추가 기여도 계산



02

EDA



- 노선별 가격.좌석 분포, 비즈니스석 도입 현황
- 시각화, 상관분석, 이상치 확인

01

데이터 구조 확인

총 300,153건, 12개 변수

수치형(int64, float64)

: index, duration, days_left, price

범주형(object)

: airline, flight, source_city, departure_time, stops, arrival_time, destination_city, class

결측치 없음

RangeIndex: 300153 entries, 0 to 300152

Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
0	index	300153 non-null	int64
1	airline	300153 non-null	object
2	flight	300153 non-null	object
3	source_city	300153 non-null	object
4	departure_time	300153 non-null	object
5	stops	300153 non-null	object
6	arrival_time	300153 non-null	object
7	destination_city	300153 non-null	object
8	class	300153 non-null	object
9	duration	300153 non-null	float64
10	days_left	300153 non-null	int64
11	price	300153 non-null	int64

dtypes: float64(1), int64(3), object(8)

01

데이터 구조 확인

상위 5개 행을 확인하여 데이터 구조와 값의 형태 이해

index	airline	flight	source_city	departure_time	stops	arrival_time	destination_city	class	duration	days_left	price
0	SpiceJet	SG-8709	Delhi	Evening	zero	Night	Mumbai	Economy	2.17	1	5953
1	SpiceJet	SG-8157	Delhi	Early_Morning	zero	Morning	Mumbai	Economy	2.33	1	5953
2	AirAsia	I5-764	Delhi	Early_Morning	zero	Early_Morning	Mumbai	Economy	2.17	1	5956
3	Vistara	UK-995	Delhi	Morning	zero	Afternoon	Mumbai	Economy	2.25	1	5955
4	Vistara	UK-963	Delhi	Morning	zero	Morning	Mumbai	Economy	2.33	1	5955

02

기초 통계량 & 분포 확인

수치형 변수 통계 요약

	count	mean	std	min	25%	50%	75%	max
index	300153.0	150076.000000	86646.852011	0.00	75038.00	150076.00	225114.00	300152.00
duration	300153.0	12.221021	7.191997	0.83	6.83	11.25	16.17	49.83
days_left	300153.0	26.004751	13.561004	1.00	15.00	26.00	38.00	49.00
price	300153.0	20889.660523	22697.767366	1105.00	4783.00	7425.00	42521.00	123071.00

02

기초 통계량 & 분포 확인

```
=== source_city ===  
source_city  
Delhi      61343  
Mumbai     60896  
Bangalore  52061  
Kolkata    46347  
Hyderabad  40806  
Chennai    38700  
Name: count, dtype: int64
```

```
=== destination_city ===  
destination_city  
Mumbai     59097  
Delhi      57360  
Bangalore  51068  
Kolkata    49534  
Hyderabad  42726  
Chennai    40368  
Name: count, dtype: int64
```

```
=== airline ===  
airline  
Vistara     127859  
Air_India   80892  
Indigo      43120  
GO_FIRST    23173  
AirAsia     16098  
SpiceJet     9011  
Name: count, dtype: int64
```

```
=== class ===  
class  
Economy     206666  
Business     93487  
Name: count, dtype: int64
```

```
=== stops ===  
stops  
one          250863  
zero         36004  
two_or_more  13286  
Name: count, dtype: int64
```

```
=== departure_time ===  
departure_time  
Morning      71146  
Early_Morning 66790  
Evening      65102  
Night        48015  
Afternoon    47794  
Late_Night   1306  
Name: count, dtype: int64
```

```
=== arrival_time ===  
arrival_time  
Night        91538  
Evening      78323  
Morning      62735  
Afternoon    38139  
Early_Morning 15417  
Late_Night   14001  
Name: count, dtype: int64
```

범주형 변수별
분포 확인

03

결측치 및 이상치 탐색

Variable	Missing Count
index	0
airline	0
flight	0
source_city	0
departure_time	0
stops	0
arrival_time	0
destination_city	0
class	0
duration	0
days_left	0
price	0

결측치 확인

모든 변수에 결측치가 없어,

별도 처리 없이 분석 가능

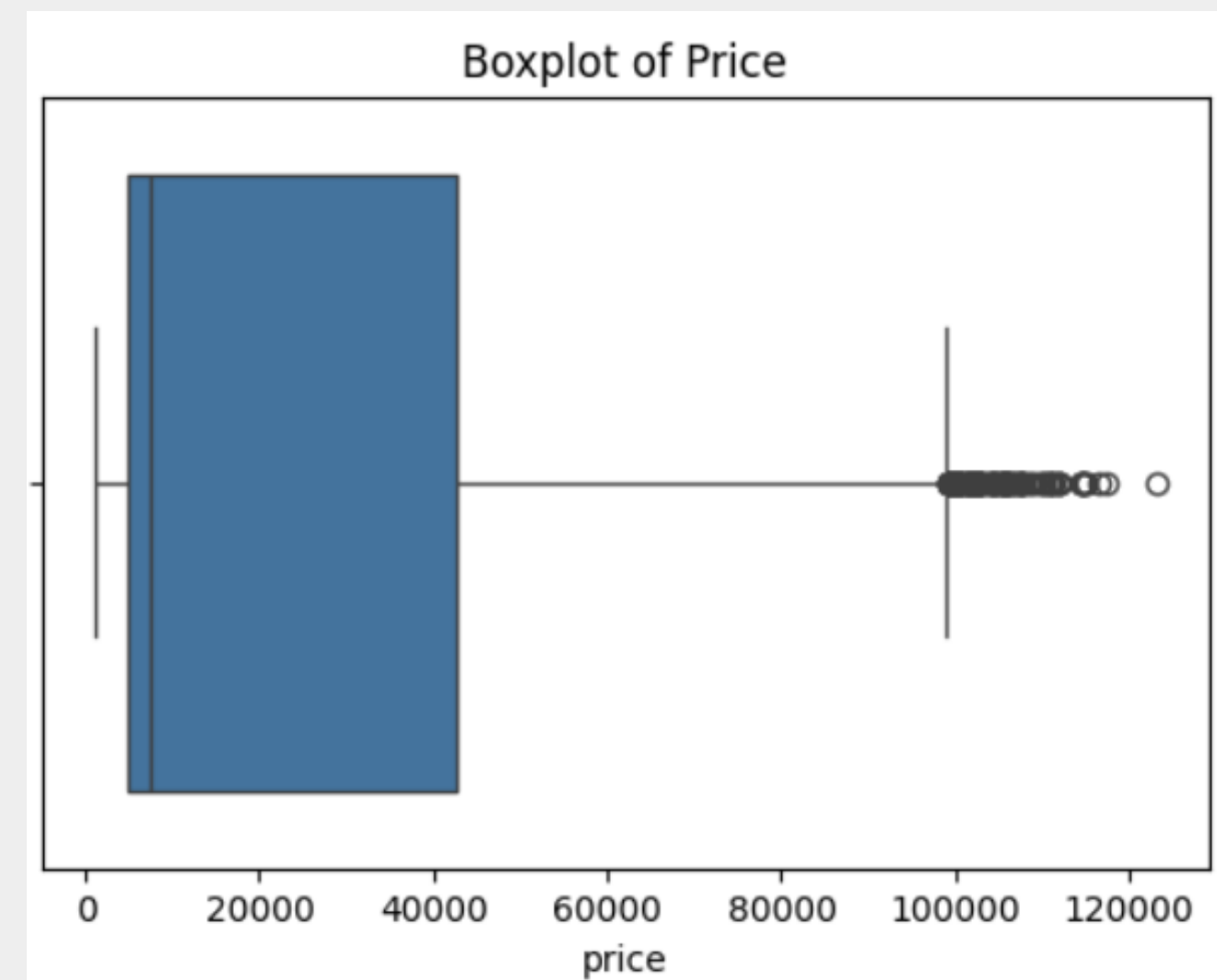
03

결측치 및 이상치 탐색

가격(price) 분포 박스플롯

일부 이상치 존재,

도메인 특성상 처리하지 않음



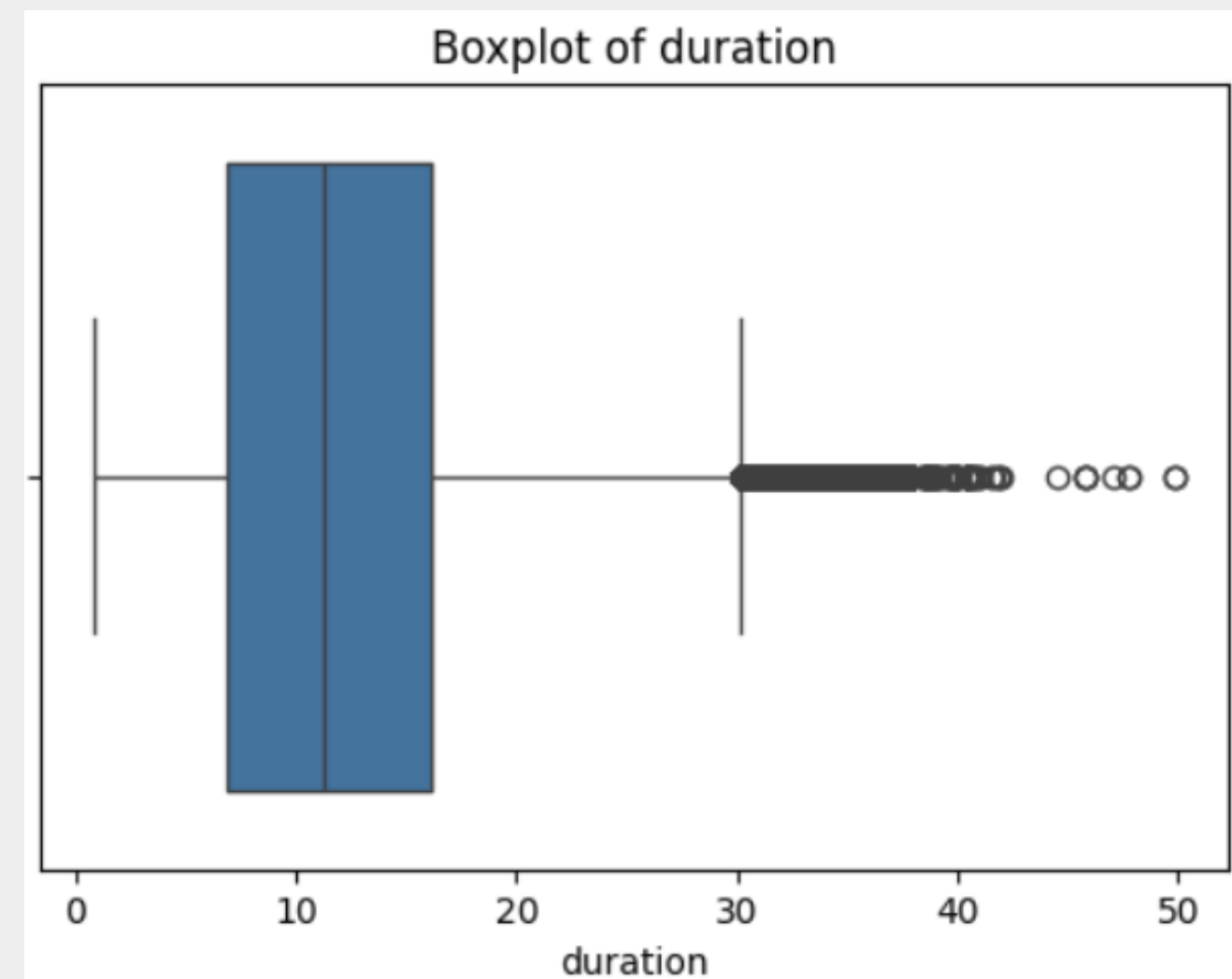
03

결측치 및 이상치 탐색

비행 시간(duration) 분포 박스플롯

일부 이상치 존재,

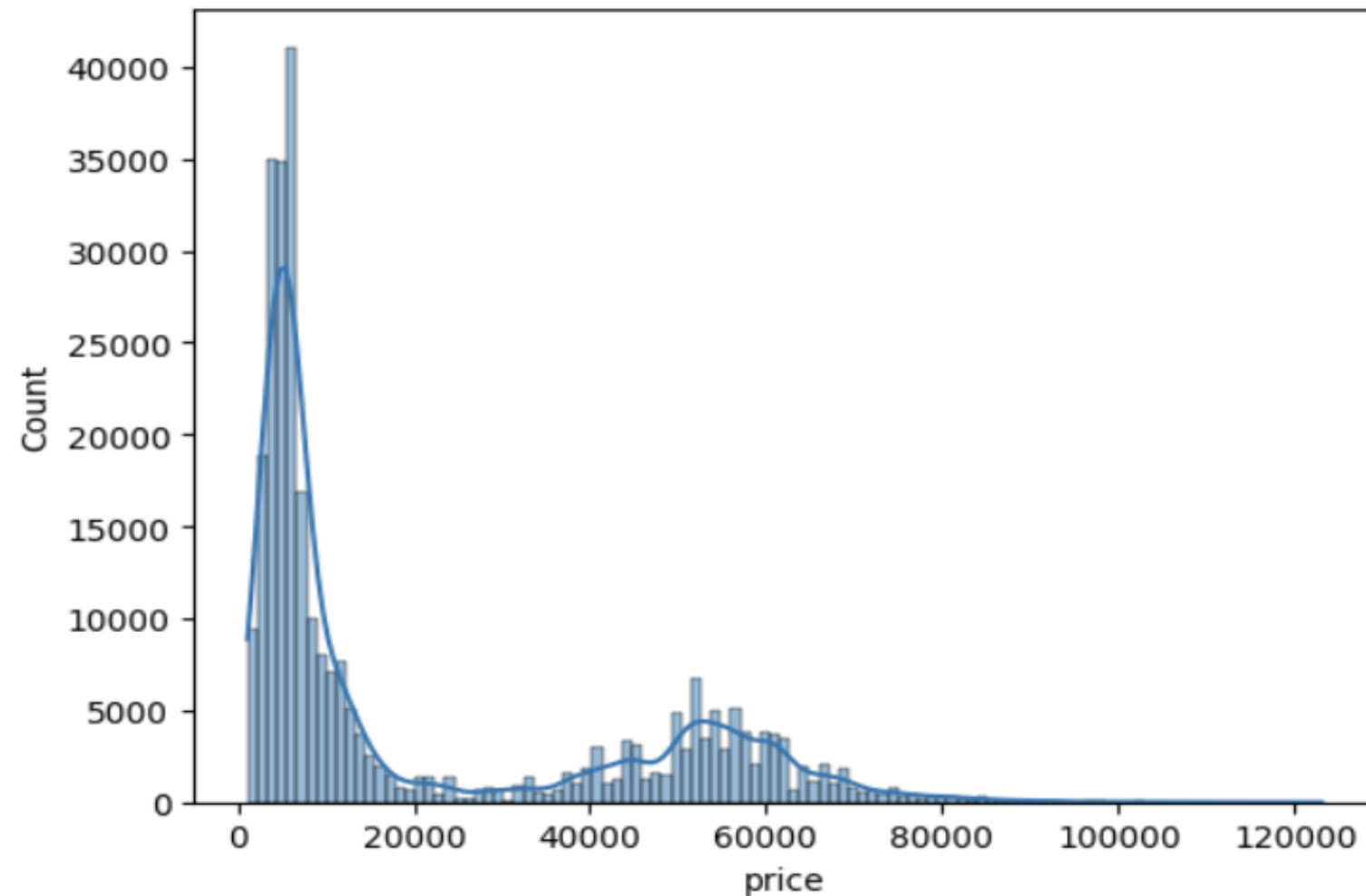
도메인 특성상 처리하지 않음



04

타깃 변수 분석 (price 같은 종속변수)

<Axes: xlabel='price', ylabel='Count'>



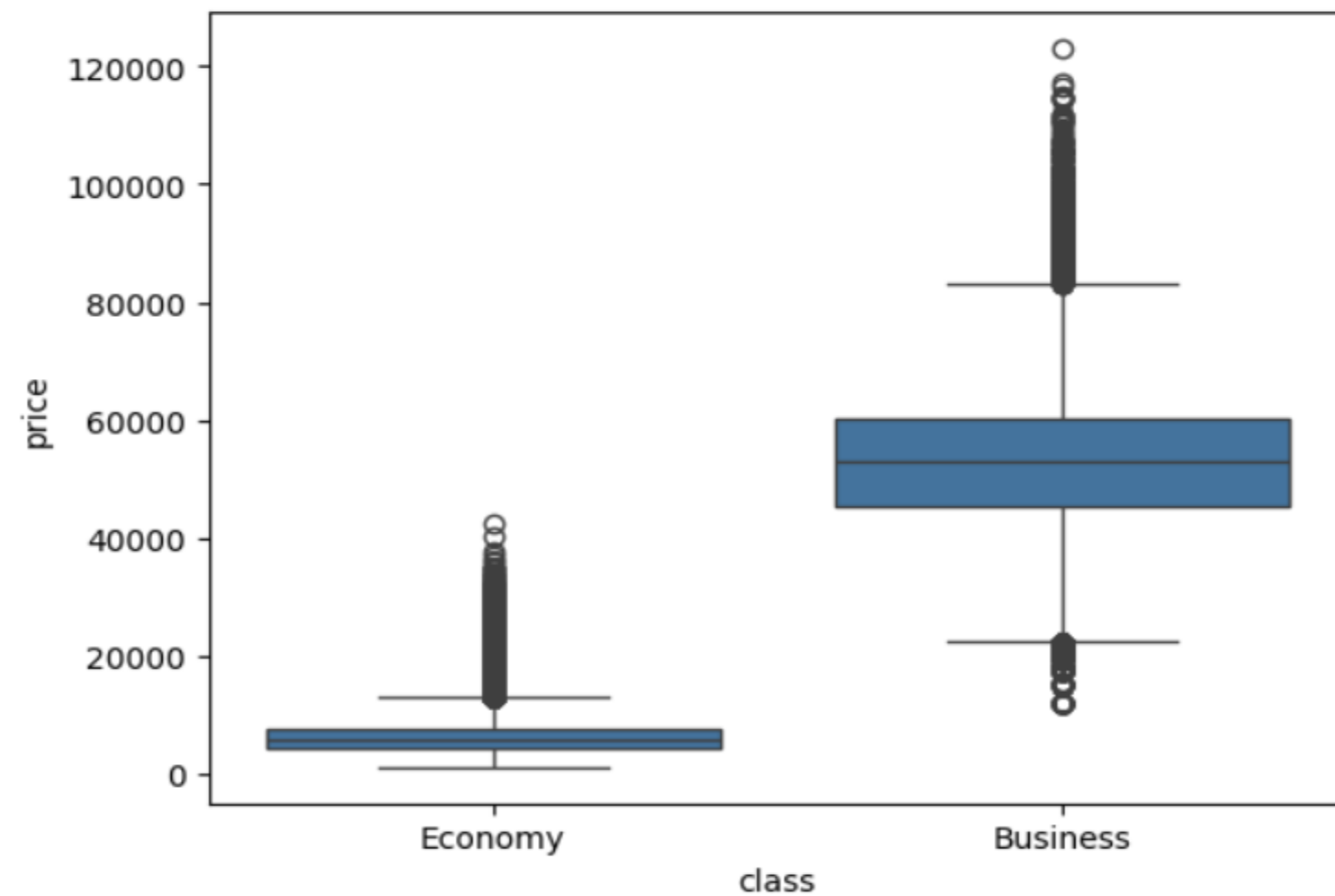
price의 분포를 확인하기 위해
히스토그램과
커널 밀도 추정(KDE) 시각화

가격 분포 오른쪽으로 치우쳐 있어 로
그 변환을 고려할 수 있지만,
모델 특성상 변환 없이 사용해도 무방

04

타깃 변수 분석 (범주형 변수 vs price)

<Axes: xlabel='class', ylabel='price'>

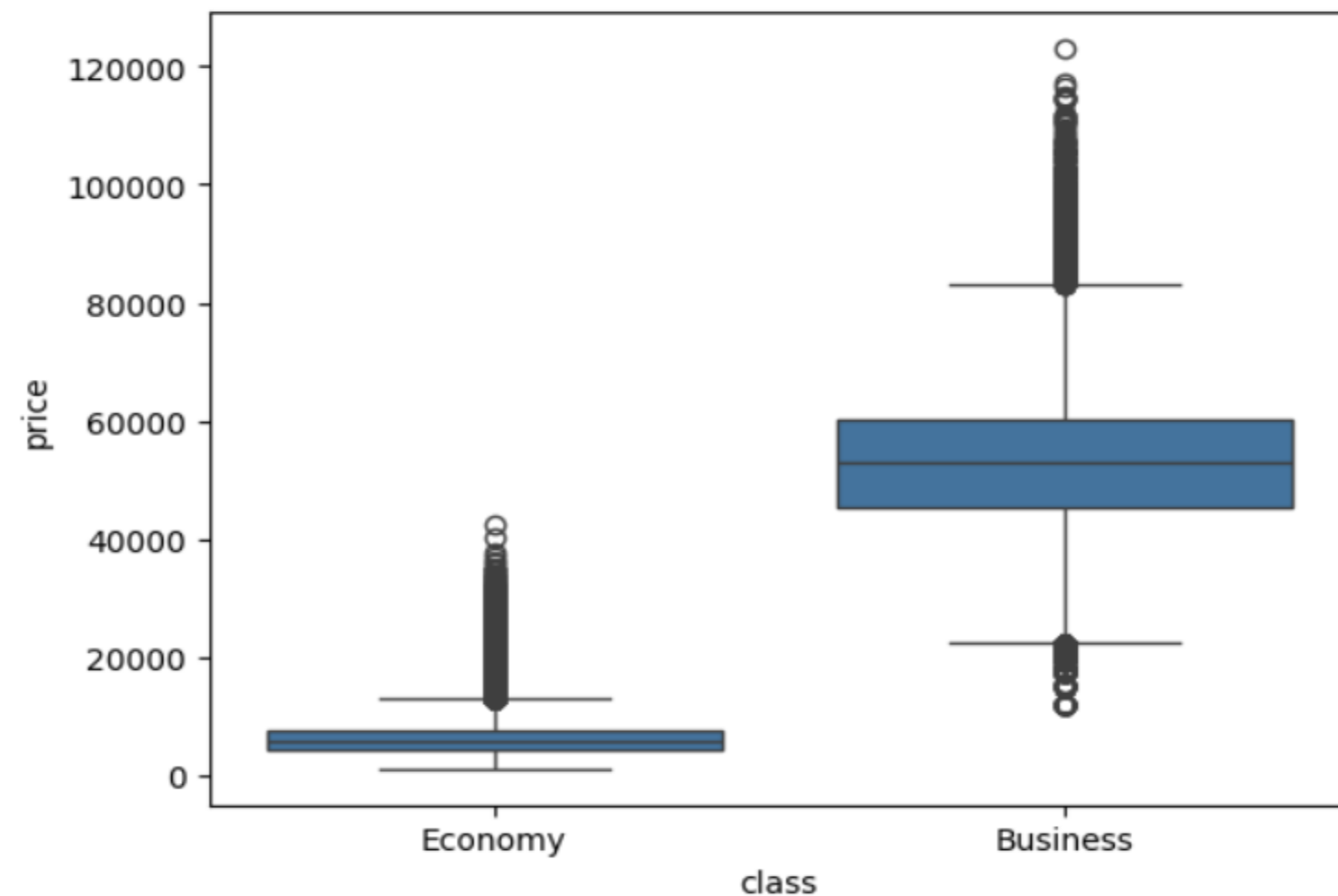


좌석 클래스(class)별
가격(price) 분포
boxplot으로 시각화

04

타깃 변수 분석 (범주형 변수 vs price)

<Axes: xlabel='class', ylabel='price'>



가격(price) 이상치 탐색

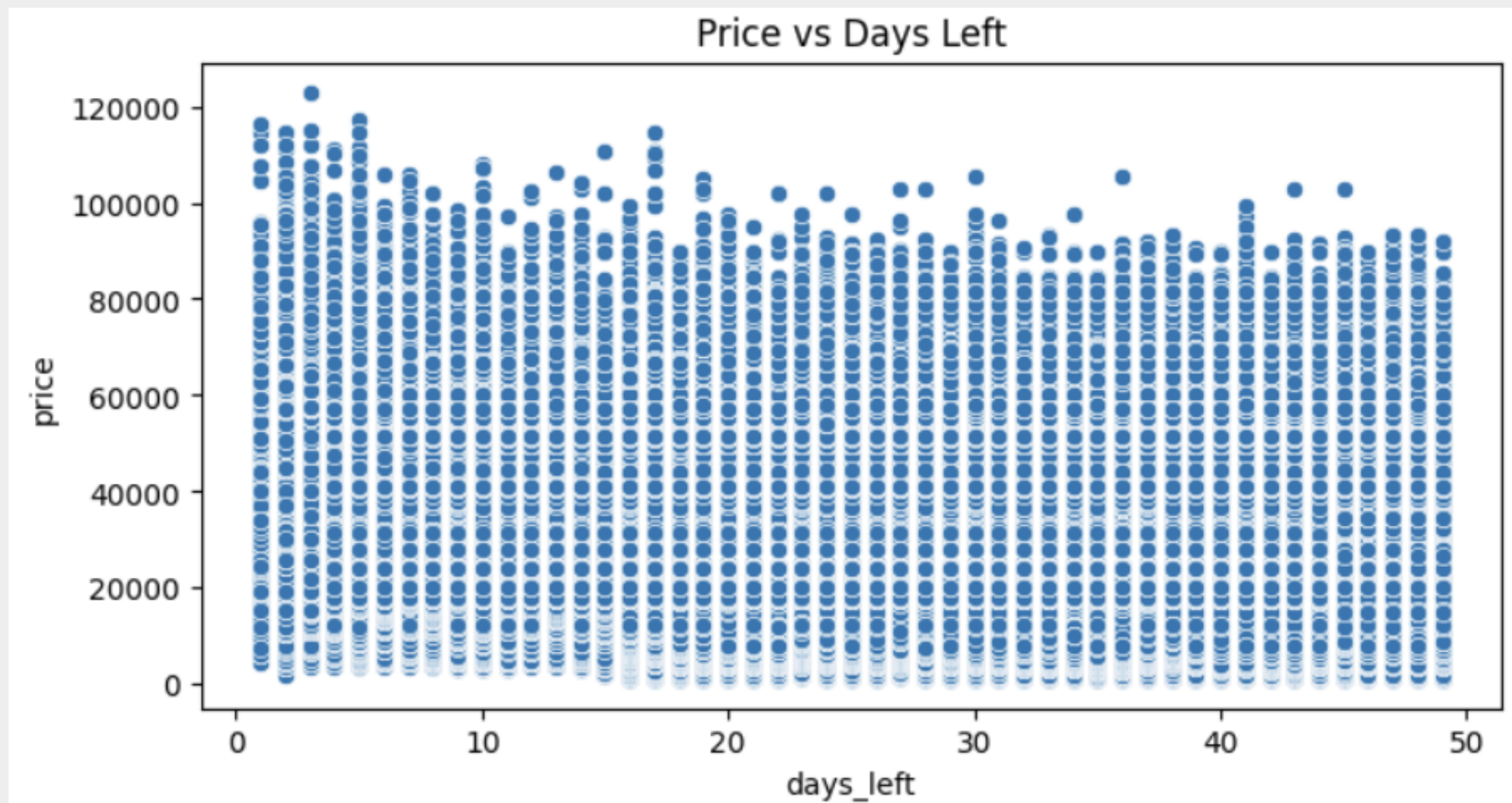
Economy : 대부분 0~15,000 사이,
일부 20,000~40,000 값이 이상치로 표시됨

Business : 중앙값 50,000~60,000,
일부 100,000 이상 값이 이상치로 표시됨

항공권 도메인의 특수성상
현실적으로 발생 가능한 값이므로,
예측 모델링 목적에서는 그대로 사용함

O4

타킷 변수 분석 (수치형 변수 vs price)



days_left와 price 간 관계
산점도/회귀선으로 확인

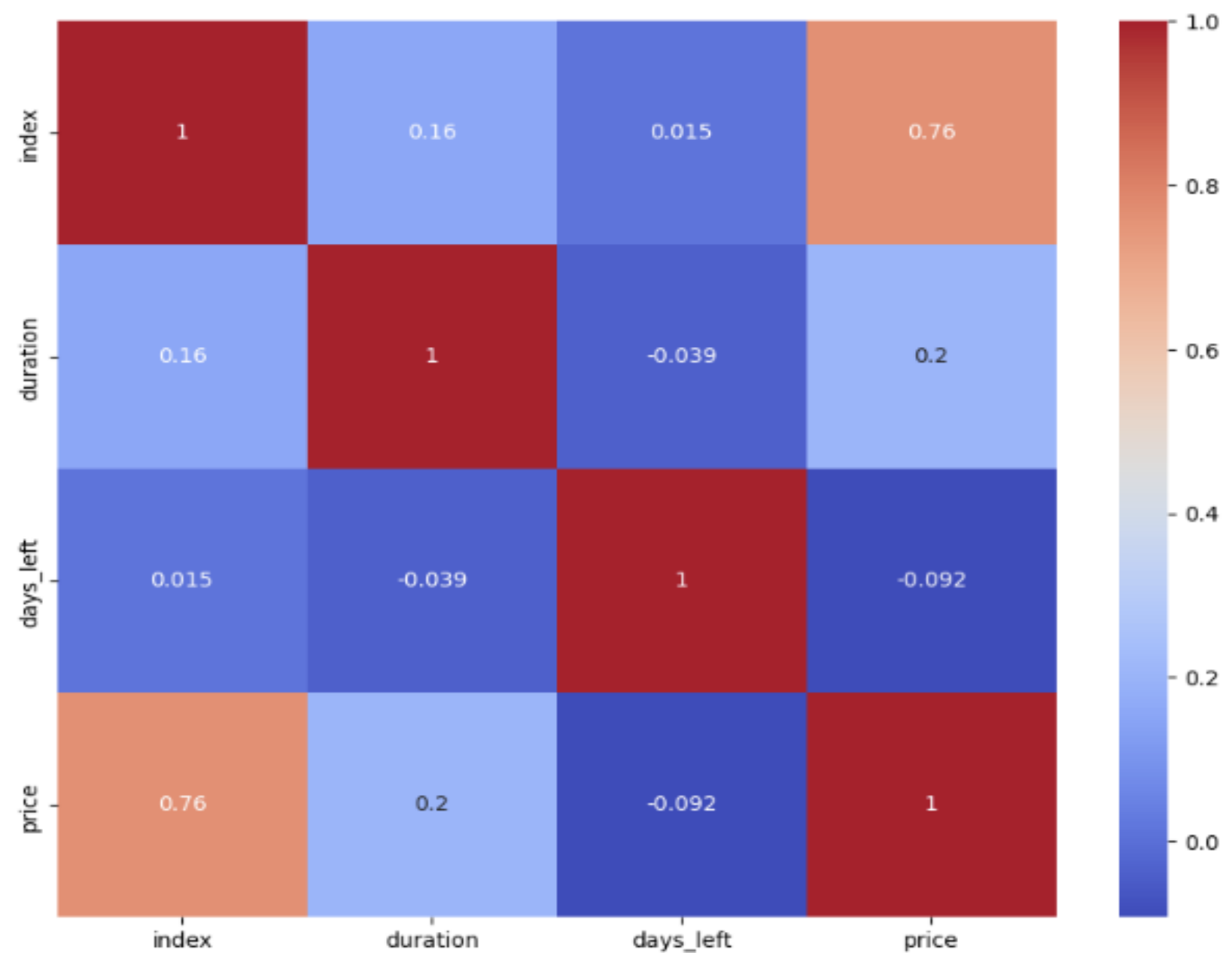
05

변수 간 관계 확인

수치형 변수 간 상관관계

(Heatmap)

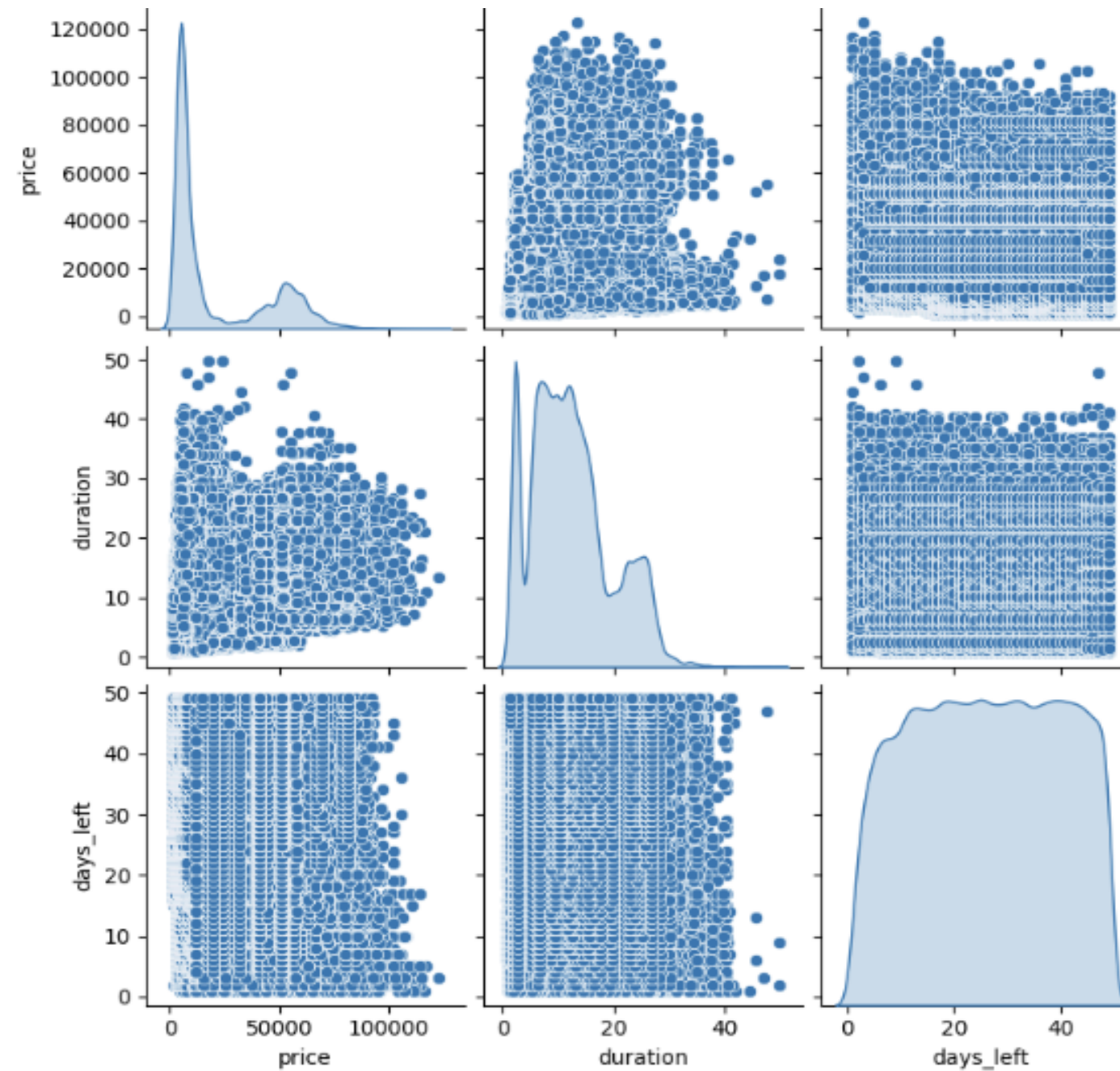
다중공선성 탐색



05

변수 간 관계 확인

수치형 변수 산점도 확인
(Scatterplot)



05

변수 간 관계 확인 (범주형 ↔ 수치형 변수 관계)

```
1 import scipy.stats as stats
2
3 # Economy와 Business의 평균 가격 차이 검정
4 economy = df[df['class'] == 'Economy']['price']
5 business = df[df['class'] == 'Business']['price']
6
7 f_stat, p_val = stats.f_oneway(economy, business)
8
9 print("F-statistic:", f_stat)
10 print("p-value:", p_val)
11
```

```
F-statistic: 2192424.594907613
p-value: 0.0
```

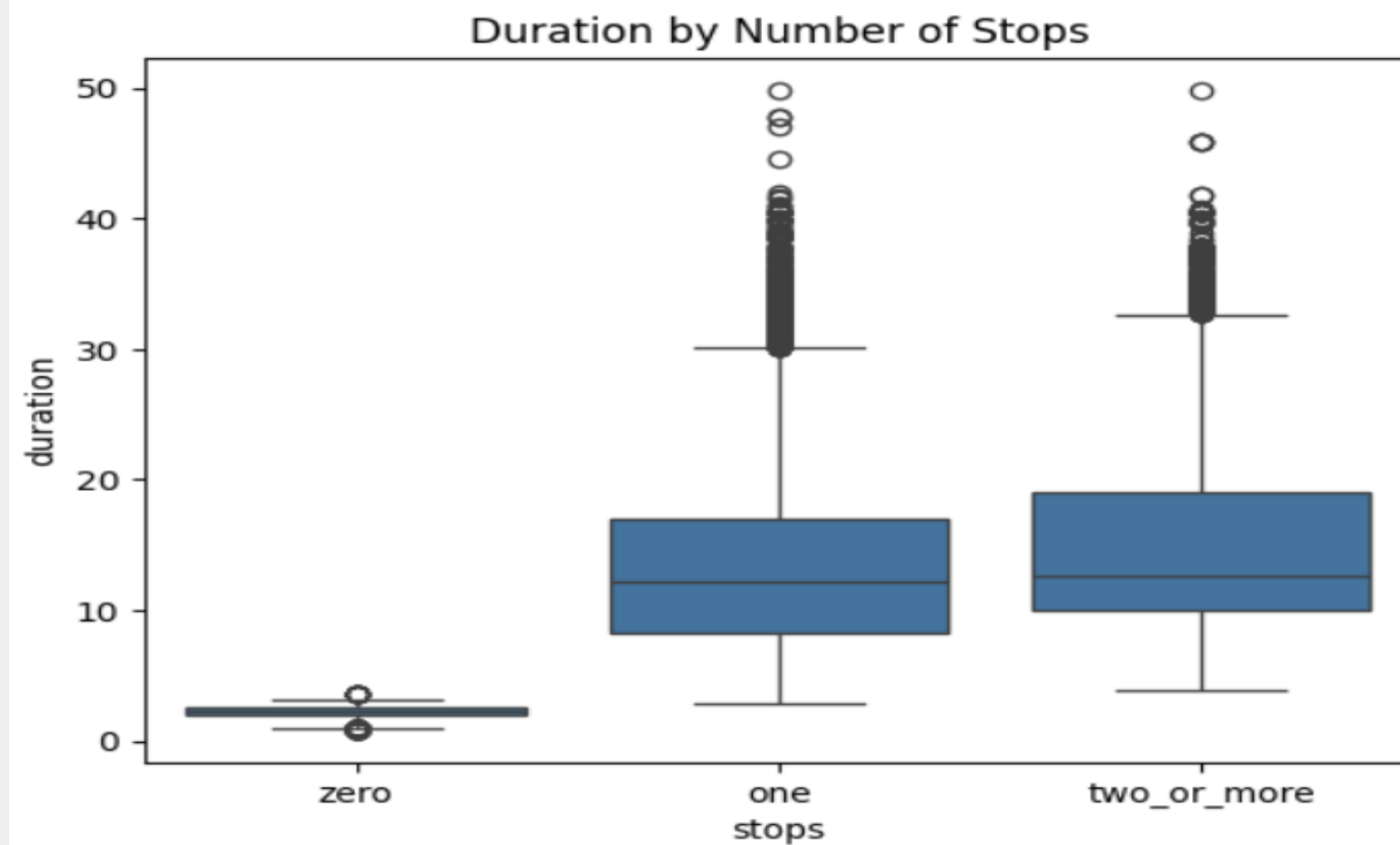
ANOVA (분산분석)

p-value < 0.05

→ Economy와 Business 클래스의 평균
가격은 통계적으로 유의미하게 다름
→ class에 따라 가격이 크게 차이 남

05

변수 간 관계 확인 (범주형 ↔ 수치형 변수 관계)



stops에 따른
duration의
분포 비교

05

변수 간 관계 확인 (범주형 ↔ 범주형 변수 관계)

교차표 & 카이제곱 검정

```
stops      one  two_or_more  zero
class
Business   84302      1083    8102
Economy    166561     12203   27902
Chi-squared statistic: 5237.069961910499
p-value: 0.0
Degrees of freedom: 2
```

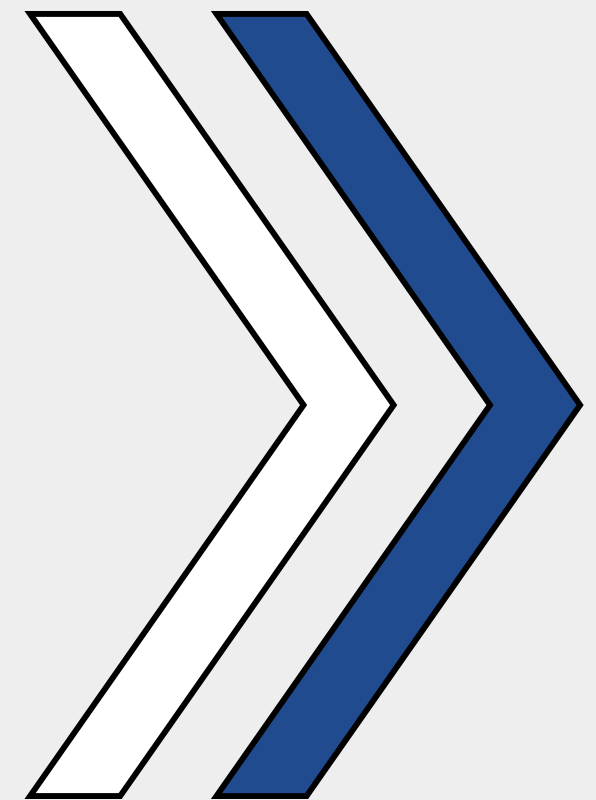
좌석 클래스(class)와
경유 횟수(stops) 간 관계
카이제곱 검정으로 확인

p-value : 0.0
두 변수는 독립이 아니며
통계적으로 유의미한 연관이 있음



03

Feature Engineering



- 컬럼제한, 변수선택
- 파생변수 생성

01

변수 선택 및 컬럼 제한

선택한 변수:

source_city

destination_city

class

price

departure_time

arrival_time

days_left

airline

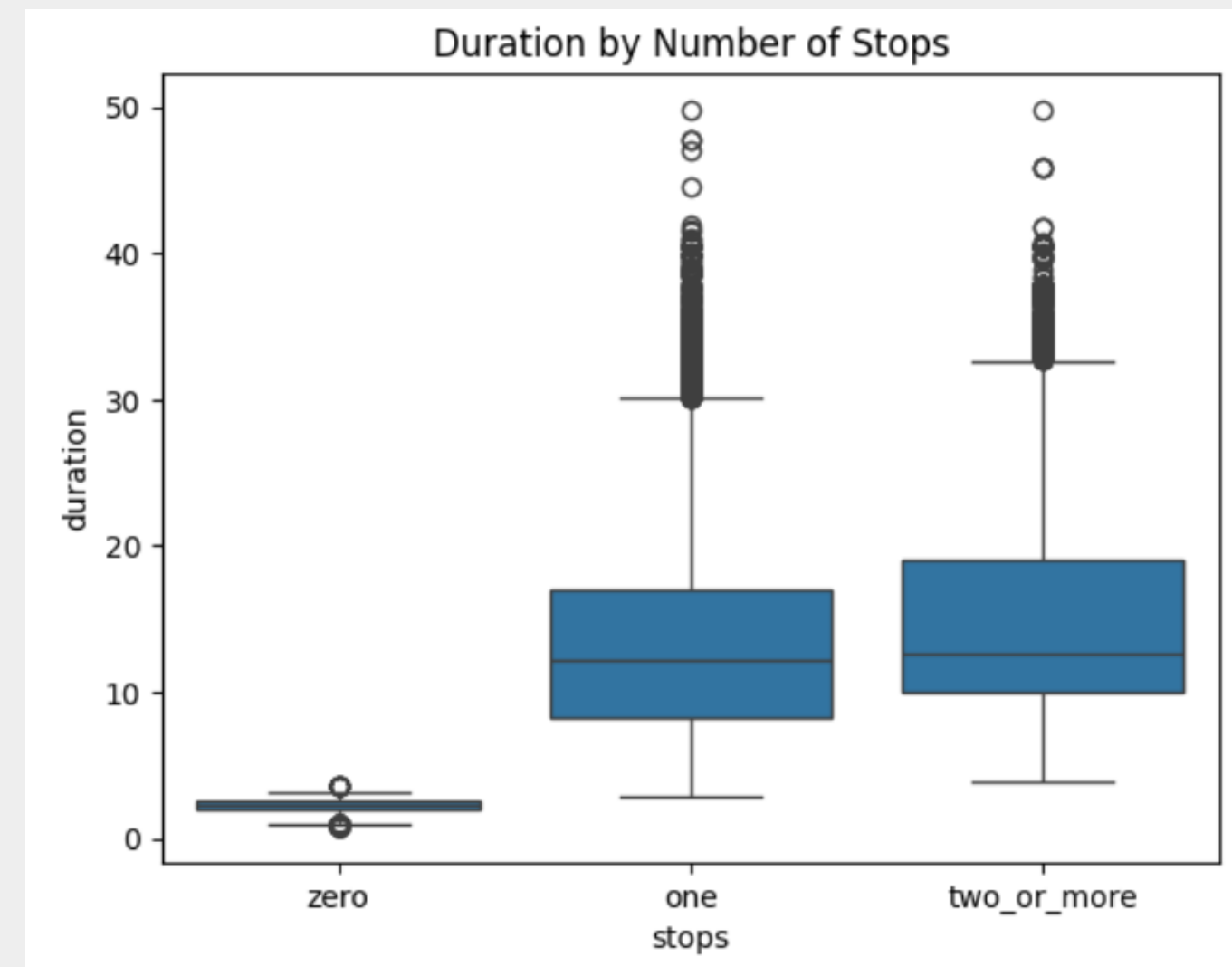
```
fsc_df= fsc_df[fsc_df['stops'] == 'zero'][[  
    'source_city', 'destination_city', 'class',  
    'price', 'departure_time', 'arrival_time', 'days_left', 'airline'  
]]
```

01

변수 선택 및 컬럼 제한

그중에서, stops==0인 Data로만 구성함

한 번 이상의 경유가 있는 경우, Outlier가 매우 많다.
왜냐하면 경유 횟수가 많을수록 이동 시간이 증가하기 때문이다.
같은 노선이라도 사람마다 각자의 사유로 인해 경유를 선택하는 경우가 발생하므로, 이런 요소들로 인한 이상치 발생을 배제하기 위해 직항인 데이터들로 구성하였다.



O2

파생변수 route 생성

새로운 파생변수 'route' 생성

route = (source city 앞 3글자) + -
+(destination city 앞 3글자)로 구성
ex) Del-Mum

이를 통해 source_city와 destination_city를
한번에 판단할 수 있고, 이 변수를 pk 요소 중
하나로 사용한다.

```
# route 파생변수 생성
```

```
fsc_df['route'] = fsc_df['source_city'].str[:3] + '-' + fsc_df['destination_city'].str[:3]
```

```
fsc_df = fsc_df.drop(columns=['source_city', 'destination_city'])
```

```
fsc_df.head()
```

03

pk 그룹화 기준 생성, 파생변수 price_mean 도입

(route) + _ + (departure_time) + _
+ (arrival_time) + _ + (class)로 pk 생성
Ex) Del_Mum_Morning_Afternoon_Economy

pk 별 항공권 가격의 평균을 계산하여 price_mean이라는 변수를 도입한다.

#3. pk 기준 만들기, 행별 price_mean 계산해서 컬럼으로 넣기

```
fsc_df['pk'] = fsc_df['route'] + '_' + fsc_df['departure_time'] + '_' + fsc_df['arrival_time'] + '_' + fsc_df['class']  
fsc_df['price_mean'] = fsc_df.groupby(['pk'])['price'].transform('mean')  
fsc_df = fsc_df.drop('price', axis=1)  
  
fsc_df.head()
```

04

좌석 수, seats, 파생변수 business_ratio

한 항공기의 total_seats = 240으로 지정(음수 방지)

business_seats : pk 중 class가 business인 것만
고려

economy_seats = total_seats - business_seats

business_ratio = business_seats / total_seats

: 총 좌석에서 비즈니스석이 차지하는 비율

```
#좌석수 컬럼 추가
fsc_df['total_seats'] = 240

# business_seats 계산: pk+class='business' 행 수
business_counts = fsc_df[fsc_df['class']=='Business'].groupby('pk').size()
fsc_df['business_seats'] = fsc_df['pk'].map(business_counts).fillna(0).astype(int)

# economy_seats 계산
fsc_df['economy_seats'] = fsc_df['total_seats'] - fsc_df['business_seats']

# business_ratio 계산
fsc_df['business_ratio'] = fsc_df['business_seats'] / fsc_df['total_seats']

fsc_df.tail()
```


05

price_premium 계산

같은 조건에서, business석의 price mean과 economy석의 price mean의 차이를 price_premium로 설정한다.

즉, $\text{price_premium} = (\text{price mean of business}) - (\text{price mean on economy})$

비즈니스석은 좌석 당 수익이 이코노미석보다 커서, 항공사 수익 구조에 영향을 준다.

또, 비즈니스석이 수익에 얼마나 기여하는지 판단할 수 있다.

```
#price_premium 계산

# 1. Business - Economy 차이 테이블 만들기
price_map = fsc_df.pivot_table(
    index=['route', 'departure_time', 'arrival_time'],
    columns='class',
    values='price_mean',
    aggfunc='first'
).fillna(0)

price_map['price_premium'] = price_map['Business'] - price_map['Economy']

# 2. 원래 fsc_df에 매핑 (row-level 구조 유지)
fsc_df['price_premium'] = fsc_df.set_index(
    ['route', 'departure_time', 'arrival_time']
).index.map(price_map['price_premium'])
```

06

파생변수 business_contribution

business_contribution
= [(business seats) * (price premium)]
/ [(total seats) * (price mean)]

분자: business석이 economy로 바뀌었을 때
생기는 추가 매출
분모: 전체 매출 규모

즉, 같은 조건에서, 전체 매출 중 business seat이 얼마나 매출에 기여하고 있는지 나타낼 수 있다.

```
#revenue 계산

import pandas as pd

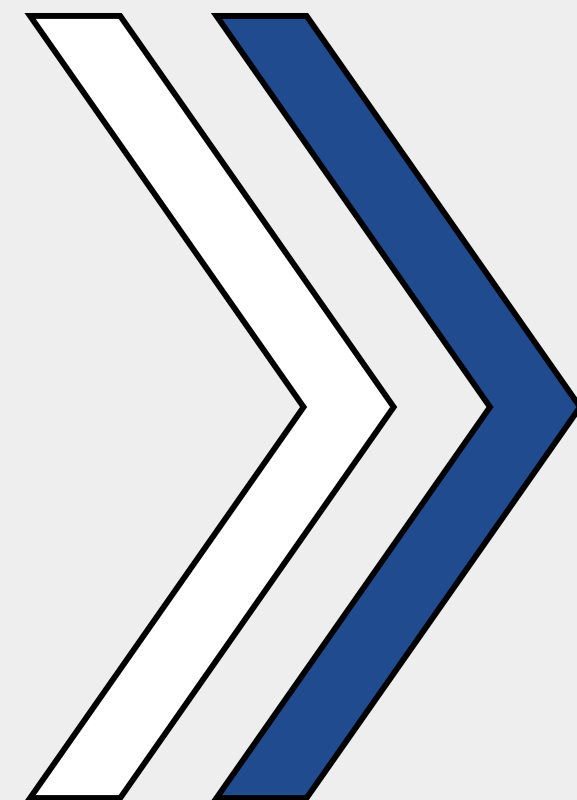
denominator = fsc_df['total_seats'] * fsc_df['price_mean']
denominator = denominator.replace(0, np.nan)
fsc_df['business_contribution'] = (fsc_df['business_seats'] * fsc_df['price_premium']) / denominator

# NaN 처리: 예를 들어 0으로 대체하거나, 분포 기반 채우기
# 여기서는 0으로 대체
fsc_df['business_contribution'] = fsc_df['business_contribution'].fillna(0)

fsc_df.tail()
```

04

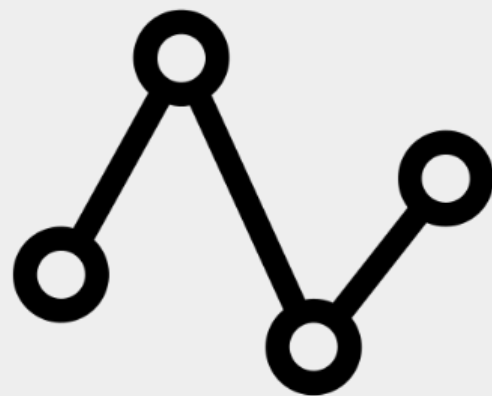
모델링 및 진단



- 모델 개요 및 데이터 준비
- 모델 학습 및 성능 비교 후 최종 모델 선택

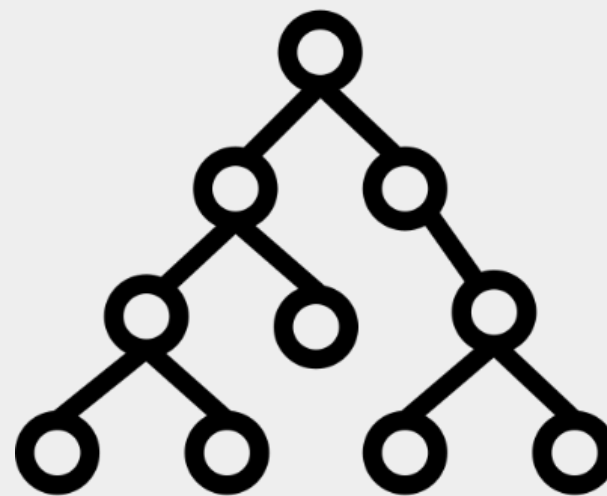
01

모델링 개요



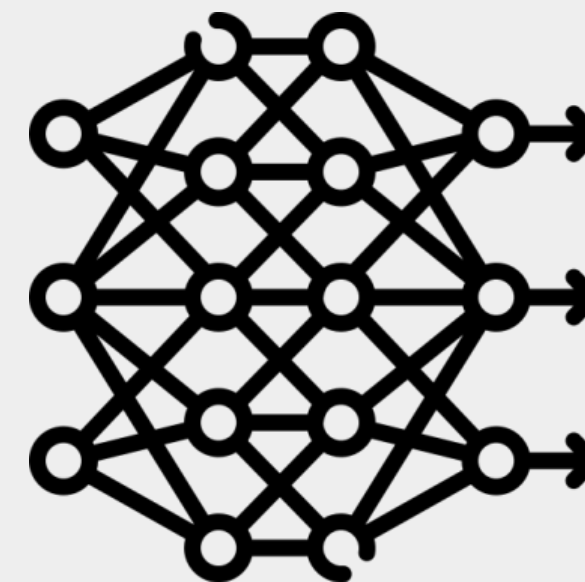
Linear Regression

가장 기본적인 선형 모델



XGBoost

비선형·상호작용 반영



ANN

복잡한 패턴 학습 가능

02

데이터 준비

```
## 독립변수 및 종속변수 정의
cat_cols = ['class', 'route', 'airline', 'departure_time', 'arrival_time'] #범주형
num_cols = ['days_left', 'business_seats', 'economy_seats', 'business_ratio'] #수치형

# 독립변수 정의
features = cat_cols + num_cols
X = pd.get_dummies(fsc_df[features], columns=cat_cols, drop_first=True) #원핫 인코딩

# 종속변수 정의
y = fsc_df['business_contribution']
```

```
# 데이터 분할 (train/test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

독립변수 및 종속변수 설정



train/test 분할

02

데이터 준비

```
# VIF 계산
vif = pd.DataFrame()
vif["Feature"] = X_const.columns
vif["VIF"] = [
    variance_inflation_factor(X_const.values, i)
    for i in range(X_const.shape[1])
]
```

- 다중공선성 진단 결과
business_seats, economy_seats,
business_ratio 간 **강한 다중공선성** 존재
→ 선형회귀 모델 해석 시 주의 필요

03

모델 학습 & 성능 비교

#모델 학습

```
LR_model = LinearRegression()  
LR_model.fit(X_train, y_train)
```

#예측 & RMSE 계산

```
y_pred = LR_model.predict(X_test)  
rmse = np.sqrt(mean_squared_error(y_test, y_pred))  
print("LR RMSE:", rmse)
```

#모델 정의 & 학습

```
xgb_model = xgb.XGBRegressor(  
    objective='reg:squarederror',  
    n_estimators=300,  
    learning_rate=0.1,  
    max_depth=6,  
    subsample=0.8,  
    colsample_bytree=0.8,  
    random_state=42  
)
```

```
xgb_model.fit(X_train, y_train)
```

#예측 & RMSE 계산

```
y_pred = xgb_model.predict(X_test)  
rmse = np.sqrt(mean_squared_error(y_test, y_pred))  
print("xgb RMSE:", rmse)
```

03

모델 학습 & 성능 비교

```
#스케일링
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

#모델 정의 & 학습

ann_model = keras.Sequential([
    keras.layers.Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dense(32, activation='relu'),
    keras.layers.Dense(1) # price 예측 (회귀니까 activation 없음)
])

ann_model.compile(optimizer='adam', loss='mse', metrics=['mae'])
```

```
ann_model.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),
    epochs=20,
    batch_size=256,
    verbose=1
)

# 예측 & RMSE 계산
y_pred = ann_model.predict(X_test)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print("ANN RMSE:", rmse)
```


04

최종 모델 선택

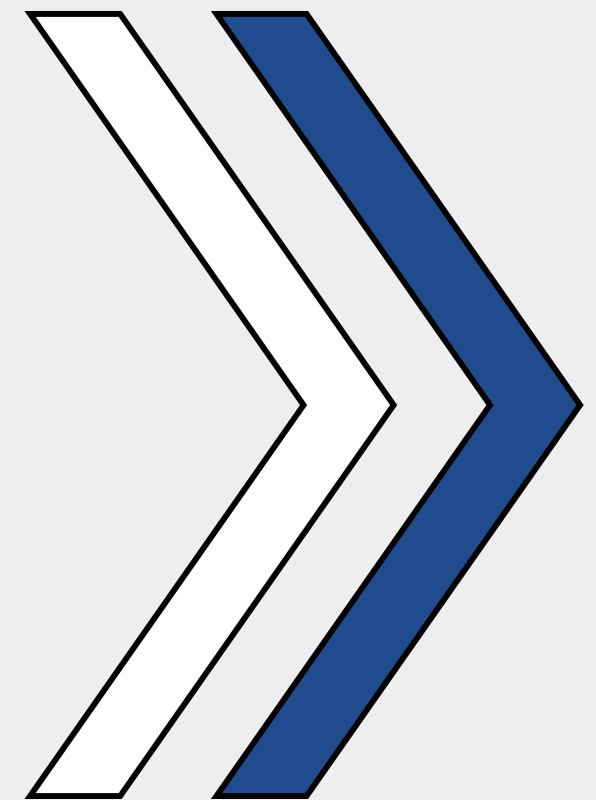
모델	RMSE
Linear Regression	0.004685
XGBoost	0.000154
ANN	0.00227

➡ 세 모델 중 RMSE가 가장 낮은 **XGBoost**을 최종 모델로 선택



05

LCC 데이터 시뮬레이션



- lcc 데이터에 모델 적용
- Optimal business seats & 예측 수익성 계산

01

LCC 데이터 모델 적용

LCC 데이터 전처리



모델 학습 데이터인 FSC와
구조를 동일하게 만들기 위한 작업

```
#lcc_df 전처리
lcc_df = df[df['airline'].isin(lcc_airlines)]
#컬럼제한
lcc_df= lcc_df[lcc_df['stops'] == 'zero'][[
    'source_city', 'destination_city', 'class',
    'price', 'departure_time', 'arrival_time', 'days_left', 'airline'
]]

#컬럼구성 동일하게
lcc_df['route'] = lcc_df['source_city'].str[:3] + '-' + lcc_df['destination_city'].str[:3]
lcc_df = lcc_df.drop(columns=['source_city', 'destination_city'])

lcc_df['pk'] = lcc_df['route'] + '_' + lcc_df['departure_time'] + '_' + lcc_df['arrival_time'] + '_' + lcc_df['class']
lcc_df['price_mean'] = lcc_df.groupby(['pk'])['price'].transform('mean')
lcc_df = lcc_df.drop('price', axis=1)
lcc_df['total_seats']=240
lcc_df['business_seats'] = 0 ✓
lcc_df['economy_seats'] = lcc_df['total_seats'] - lcc_df['business_seats']
lcc_df['business_ratio'] = lcc_df['business_seats'] / lcc_df['total_seats']
lcc_df['price_premium'] = 0 ✓
```


01

LCC 데이터 모델 적용

<fsc_df>

Column	Non-Null Count	Dtype
class	10260	object
departure_time	10260	object
arrival_time	10260	object
days_left	10260	int64
airline	10260	object
duration	10260	float64
route	10260	object
pk	10260	object
price_mean	10260	float64
total_seats	10260	int64
business_seats	10260	int64
economy_seats	10260	int64
business_ratio	10260	float64
price_premium	10260	float64
business_contribution	10260	float64

<lcc_df>

Column	Non-Null Count	Dtype
class	10260	object
departure_time	10260	object
arrival_time	10260	object
days_left	10260	int64
airline	10260	object
duration	10260	float64
route	10260	object
pk	10260	object
price_mean	10260	float64
total_seats	10260	int64
business_seats	10260	int64
economy_seats	10260	int64
business_ratio	10260	float64
price_premium	10260	float64
business_contribution	10260	float64

02

XGboost을 이용한 Grid Search 기반의 시뮬레이션 최적화

<시뮬레이션 과정>

group_cols 조합에 대해 각각 첫 행 잡기



business_seats를 0부터 240까지 20씩 증가 시키며 XGboost 모델 적용



예측값을 뽑아 저장



첫번째 peak를 현실적인 추정값으로 설정



각 group_cols 조합에서 최적 찾기

O2

XGboost을 이용한 Grid Search 기반의 시뮬레이션 최적화

1. group_cols 조합에 대해 각각 첫 행 잡기

```
group_cols = ['route', 'arrival_time', 'departure_time', 'class', 'days_left', 'airline', 'price_mean']
groups = lcc_df[group_cols].drop_duplicates()

all_results = []

for _, grp in groups.iterrows():
    # 해당 조건을 만족하는 첫 번째 항공편 뽑기
    sample_row = lcc_df[
        (lcc_df['route'] == grp['route']) &
        (lcc_df['arrival_time'] == grp['arrival_time']) &
        (lcc_df['departure_time'] == grp['departure_time']) &
        (lcc_df['class'] == grp['class']) &
        (lcc_df['days_left'] == grp['days_left']) &
        (lcc_df['airline'] == grp['airline']) &
        (lcc_df['price_mean'] == grp['price_mean'])
    ].iloc[0].copy()
```

2. business_seats를 0부터 240까지 20씩 증가 시킴

```
# 20씩 단위로 시뮬레이션
for bs in range(0, sample_row['total_seats']+1, 20):
    eco = sample_row['total_seats'] - bs
    ratio = bs / sample_row['total_seats']

    features = pd.DataFrame([
        'class': sample_row['class'],
        'airline': sample_row['airline'],
        'route': sample_row['route'],
        'departure_time': sample_row['departure_time'],
        'arrival_time': sample_row['arrival_time'],
        'days_left': sample_row['days_left'],
        'business_seats': bs,
        'economy_seats': eco,
        'business_ratio': ratio
    ])
```

02

XGboost을 이용한 Grid Search 기반의 시뮬레이션 최적화

3. 인코딩 후 XG boost 모델 적용 → 최종 결과 데이터 도출

```
# 인코딩
X_cat = pd.get_dummies(features[cat_cols], drop_first=True)
X_num = features[num_cols]
X_feat = pd.concat([X_num, X_cat], axis=1).reindex(columns=X_columns, fill_value=0)

# XG boost 예측
y_pred = xgb_model.predict(X_feat)[0]

# 결과 저장
all_results.append((
    grp['route'], grp['arrival_time'], grp['departure_time'], grp['class'],
    bs, y_pred
))

# 결과 DataFrame
whatif_df = pd.DataFrame(all_results, columns=['route', 'arrival_time', 'departure_time', 'class', 'business_seats', 'predicted_profit'])
whatif_df.to_excel('whatif_df.xlsx', index=False)
```

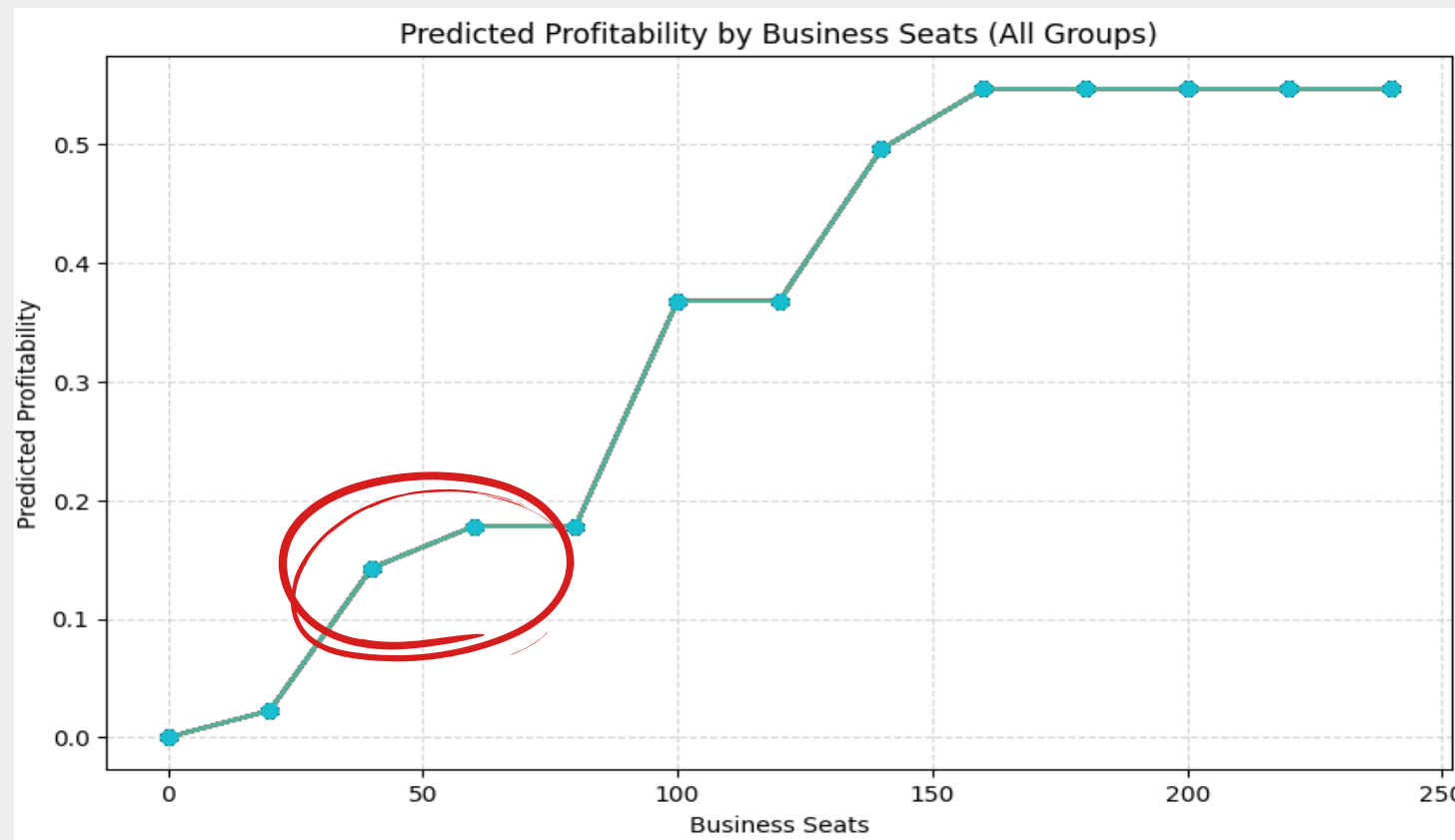
<결과 예시>

route	arrival_time	departure_time	class	business_seats	predicted_profit
Del-Mum	Night	Evening	Economy	0	9.56408E-05
Del-Mum	Night	Evening	Economy	20	0.022535371
Del-Mum	Night	Evening	Economy	40	0.142694622
Del-Mum	Night	Evening	Economy	60	0.177993566
Del-Mum	Night	Evening	Economy	80	0.177993566
Del-Mum	Night	Evening	Economy	100	0.367880613
Del-Mum	Night	Evening	Economy	120	0.367880613
Del-Mum	Night	Evening	Economy	140	0.495964944
Del-Mum	Night	Evening	Economy	160	0.54683584
Del-Mum	Night	Evening	Economy	180	0.54683584
Del-Mum	Night	Evening	Economy	200	0.54683584
Del-Mum	Night	Evening	Economy	220	0.54683584
Del-Mum	Night	Evening	Economy	240	0.54683584

02

XGboost을 이용한 Grid Search 기반의 시뮬레이션 최적화

4. 첫번째 plat를 현실적인 추정값으로 설정



- (Ban-Che, 'Early_Morning', 'Early_Morning', 'Economy')
- (Ban-Del, 'Late_Night', 'Night', 'Economy')
- (Ban-Del, 'Morning', 'Early_Morning', 'Economy')
- (Ban-Del, 'Night', 'Evening', 'Economy')
- (Ban-Del, 'Night', 'Night', 'Economy')
- (Ban-Hyd, 'Evening', 'Evening', 'Economy')
- (Ban-Hyd, 'Morning', 'Early_Morning', 'Economy')
- (Ban-Kol, 'Early_Morning', 'Early_Morning', 'Economy')
- (Ban-Kol, 'Evening', 'Evening', 'Economy')
- (Ban-Kol, 'Morning', 'Early_Morning', 'Economy')
- (Ban-Kol, 'Night', 'Evening', 'Economy')
- (Ban-Mum, 'Early_Morning', 'Early_Morning', 'Economy')
- (Ban-Mum, 'Morning', 'Early_Morning', 'Economy')
- (Ban-Mum, 'Night', 'Evening', 'Economy')
- (Che-Ban, 'Morning', 'Early_Morning', 'Economy')
- (Che-Ban, 'Morning', 'Morning', 'Economy')
- (Che-Del, 'Late_Night', 'Night', 'Economy')
- (Che-Del, 'Morning', 'Early_Morning', 'Economy')
- (Che-Hyd, 'Early_Morning', 'Early_Morning', 'Economy')
- (Che-Kol, 'Morning', 'Early_Morning', 'Economy')
- (Che-Kol, 'Night', 'Evening', 'Economy')
- (Che-Mum, 'Early_Morning', 'Early_Morning', 'Economy')
- (Che-Mum, 'Morning', 'Early_Morning', 'Economy')
- (Che-Mum, 'Night', 'Night', 'Economy')
- (Del-Ban, 'Early_Morning', 'Early_Morning', 'Economy')
- (Del-Ban, 'Evening', 'Evening', 'Economy')
- (Del-Ban, 'Late_Night', 'Night', 'Economy')
- (Del-Ban, 'Morning', 'Early_Morning', 'Economy')
- (Del-Ban, 'Night', 'Evening', 'Economy')
- (Del-Ban, 'Night', 'Night', 'Economy')
- (Del-Che, 'Morning', 'Early_Morning', 'Economy')
- (Del-Che, 'Night', 'Evening', 'Economy')
- (Del-Hyd, 'Night', 'Evening', 'Economy')
- (Del-Kol, 'Morning', 'Early_Morning', 'Economy')
- (Del-Kol, 'Morning', 'Morning', 'Economy')
- (Del-Kol, 'Night', 'Evening', 'Economy')
- (Del-Kol, 'Night', 'Night', 'Economy')
- (Del-Mum, 'Morning', 'Early_Morning', 'Economy')
- (Del-Mum, 'Morning', 'Morning', 'Economy')
- (Del-Mum, 'Night', 'Evening', 'Economy')
- (Hyd-Che, 'Afternoon', 'Afternoon', 'Economy')
- (Hyd-Del, 'Night', 'Night', 'Economy')
- (Hyd-Kol, 'Night', 'Evening', 'Economy')
- (Kol-Ban, 'Late_Night', 'Night', 'Economy')
- (Kol-Ban, 'Morning', 'Early_Morning', 'Economy')
- (Kol-Ban, 'Night', 'Evening', 'Economy')
- (Kol-Ban, 'Night', 'Night', 'Economy')
- (Kol-Che, 'Afternoon', 'Morning', 'Economy')
- (Kol-Che, 'Evening', 'Afternoon', 'Economy')
- (Kol-Che, 'Night', 'Night', 'Economy')
- (Kol-Del, 'Late_Night', 'Night', 'Economy')
- (Kol-Del, 'Morning', 'Early_Morning', 'Economy')
- (Kol-Del, 'Night', 'Evening', 'Economy')
- (Kol-Del, 'Night', 'Night', 'Economy')
- (Kol-Mum, 'Afternoon', 'Morning', 'Economy')
- (Kol-Mum, 'Late_Night', 'Night', 'Economy')
- (Kol-Mum, 'Morning', 'Morning', 'Economy')
- (Kol-Mum, 'Night', 'Evening', 'Economy')
- (Mum-Ban, 'Night', 'Night', 'Economy')
- (Mum-Che, 'Early_Morning', 'Early_Morning', 'Economy')
- (Mum-Che, 'Night', 'Evening', 'Economy')
- (Mum-Che, 'Night', 'Night', 'Economy')
- (Mum-Del, 'Afternoon', 'Morning', 'Economy')
- (Mum-Del, 'Morning', 'Early_Morning', 'Economy')
- (Mum-Del, 'Morning', 'Morning', 'Economy')
- (Mum-Del, 'Night', 'Evening', 'Economy')
- (Mum-Del, 'Night', 'Night', 'Economy')
- (Mum-Kol, 'Morning', 'Early_Morning', 'Economy')
- (Mum-Kol, 'Night', 'Evening', 'Economy')

02

XGboost을 이용한 Grid Search 기반의 시뮬레이션 최적화

5. 각 group_cols 조합에서 최적 찾기

```
group_cols = ['route', 'departure_time', 'arrival_time', 'class']

def first_flat_row(group):
    profits = group['predicted_profit'].values
    for i in range(len(profits)-1):
        if profits[i] == profits[i+1]: # flat 시작점
            return group.iloc[i]      # flat 시작 행 반환
    return None # flat이 없는 경우

# 그룹별로 적용
flat_rows = whatif_df.groupby(group_cols, group_keys=False).apply(lambda g: first_flat_row(g))

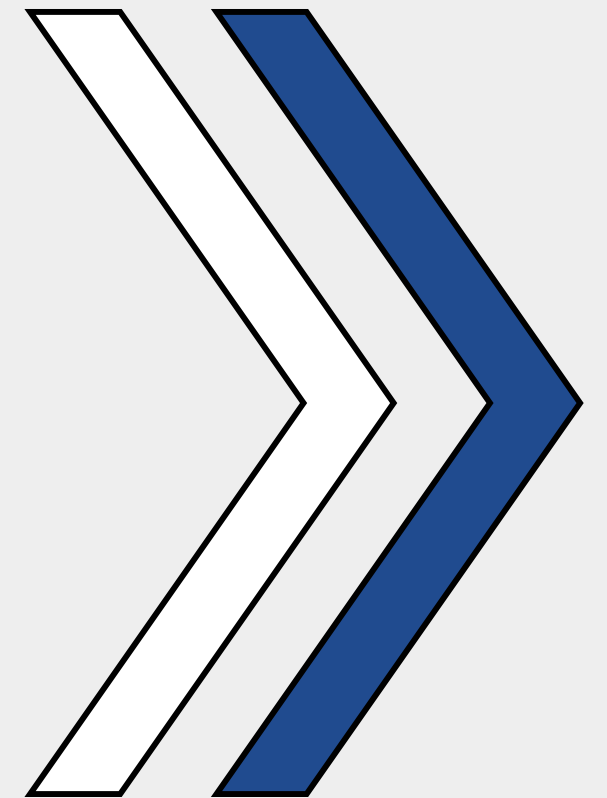
# None 제거
flat_rows = flat_rows.dropna(how='all')

# 확인
print(flat_rows[['route', 'departure_time', 'arrival_time', 'class', 'business_seats', 'predicted_profit']])
```



06

결과 해석 및 결론



- preedicted_profitability, optimal business seats 결과 분석
- 시사점

01

predicted_profitability, optimal business seats 결과 분석

Ban-Che	Early_Morning	Early_Morning	Economy	60	Che-Mum	Early_Morning	Early_Morning	Economy	60	Kol-Ban	Morning	Early_Morning	Economy	60
Ban-Del	Morning	Early_Morning	Economy	60	Che-Mum	Morning	Early_Morning	Economy	60	Kol-Ban	Night	Evening	Economy	60
Ban-Del	Night	Evening	Economy	60	Che-Mum	Night	Night	Economy	60	Kol-Ban	Late_Night	Night	Economy	60
Ban-Del	Late_Night	Night	Economy	60	Del-Ban	Early_Morning	Early_Morning	Economy	60	Kol-Ban	Night	Night	Economy	60
Ban-Del	Night	Night	Economy	60	Del-Ban	Morning	Early_Morning	Economy	60	Kol-Che	Evening	Afternoon	Economy	60
Ban-Hyd	Early_Morning	Early_Morning	Economy	60	Del-Ban	Evening	Evening	Economy	60	Kol-Che	Afternoon	Morning	Economy	60
Ban-Hyd	Morning	Early_Morning	Economy	60	Del-Ban	Night	Evening	Economy	60	Kol-Che	Night	Night	Economy	60
Ban-Hyd	Evening	Evening	Economy	60	Del-Ban	Late_Night	Night	Economy	60	Kol-Del	Morning	Early_Morning	Economy	60
Ban-Kol	Early_Morning	Early_Morning	Economy	60	Del-Ban	Night	Night	Economy	60	Kol-Del	Night	Evening	Economy	60
Ban-Kol	Morning	Early_Morning	Economy	60	Del-Ban	Night	Night	Economy	60	Kol-Del	Late_Night	Night	Economy	60
Ban-Kol	Evening	Evening	Economy	60	Del-Che	Morning	Early_Morning	Economy	60	Kol-Del	Night	Night	Economy	60
Ban-Kol	Night	Evening	Economy	60	Del-Che	Night	Evening	Economy	60	Kol-Mum	Night	Evening	Economy	60
Ban-Mum	Early_Morning	Early_Morning	Economy	60	Del-Hyd	Night	Evening	Economy	60	Kol-Mum	Afternoon	Morning	Economy	60
Ban-Mum	Morning	Early_Morning	Economy	60	Del-Kol	Morning	Early_Morning	Economy	60	Kol-Mum	Morning	Morning	Economy	60
Ban-Mum	Night	Evening	Economy	60	Del-Kol	Night	Evening	Economy	60	Kol-Mum	Late_Night	Night	Economy	60
Che-Ban	Morning	Early_Morning	Economy	60	Del-Kol	Morning	Morning	Economy	60	Mum-Ban	Night	Night	Economy	60
Che-Ban	Morning	Morning	Economy	60	Del-Kol	Night	Night	Economy	60	Mum-Che	Early_Morning	Early_Morning	Economy	60
Che-Del	Morning	Early_Morning	Economy	60	Del-Mum	Morning	Early_Morning	Economy	60	Mum-Che	Night	Evening	Economy	60
Che-Del	Late_Night	Night	Economy	60	Del-Mum	Night	Evening	Economy	60	Mum-Che	Night	Night	Economy	60
Che-Hyd	Early_Morning	Early_Morning	Economy	60	Del-Mum	Morning	Morning	Economy	60	Mum-Del	Morning	Early_Morning	Economy	60
Che-Kol	Morning	Early_Morning	Economy	60	Hyd-Che	Afternoon	Afternoon	Economy	60	Mum-Del	Night	Evening	Economy	60
Che-Kol	Night	Evening	Economy	60	Hyd-Del	Night	Night	Economy	60	Mum-Del	Afternoon	Morning	Economy	60
					Hvd-Kol	Night	Evening	Economy	60	Mum-Del	Morning	Morning	Economy	60
										Mum-Del	Night	Night	Economy	60
										Mum-Kol	Morning	Early_Morning	Economy	60
										Mum-Kol	Night	Evening	Economy	60

02 시사점

"현실과의 차이점"

실제 국내선 항공기는 보통 170~180석
그중 비즈니스석은 20석 이내

비즈니스석 비율 → 전체 5~10%

음수값 조정을 위해 240석으로 상향 조정
분석 결과값=보통 60석 도입

비즈니스석 비율 → 전체 25%

실제보다 비즈니스석 비율을 높게 설정하는 것이 수익성 개선에 도움

감사합니다